

Trabajo práctico — Patrones de interfaz con DOM y eventos

Librería del Sur— Sistema interno de gestión

CONTEXTO DEL PROBLEMA

Sos el único desarrollador frontend de Librería del Sur, una librería independiente de Ushuaia que quiere lanzar su catálogo digital. El equipo de diseño ya entregó la maqueta y recibís el código HTML y CSS base listo. Tu trabajo es darle vida con JavaScript: manipular el DOM, responder a eventos del usuario e implementar los patrones de interfaz que el diseño requiere. El código base ya está escrito — no hace falta modificar el HTML ni el CSS. Solo agregás JavaScript en `app.js`

Catálogo con filtros y vista de detalle

Implementá las dos funciones que faltan en `app.js` para que el catálogo funcione: `renderGrilla()` que construye las tarjetas con `createElement`, e `initFiltros()` que filtra por categoría al hacer clic en los botones.

Requerimientos

- 01 `renderGrilla(lista)` recibe un array de libros y por cada uno crea una tarjeta con `createElement` e `appendChild` — sin usar `innerHTML`.
- 02 Cada tarjeta muestra: título, autor, categoría (como badge) y precio formateado en pesos (ej. "\$4.200").
- 03 `initFiltros()` escucha el evento click en cada botón de filtro; al activarse, marca el botón con la clase `.activo` y vuelve a renderizar solo los libros de esa categoría.
- 04 Al hacer clic en una tarjeta, la modal se abre con el título, autor, descripción completa y precio del libro seleccionado.
- 05 La modal se cierra con el botón 5, con clic en el overlay y con la tecla `Escape`.

Criterios de evaluación: ¿cómo se que está bien?

DOM

- Uso correcto de `createElement`
- Nodos bien anidados
- Limpieza del contenedor antes de re-render

Eventos

- `addEventListener` en cada trigger
- Modal cierra con 3 métodos
- Estado activo del filtro visible

Agregar carrito con patrón wizard

Partiendo del ejercicio anterior resuelto, extendé la aplicación con un carrito de compras y un proceso de checkout de 3 pasos implementado con el patrón Wizard.

Requerimientos

- 01 Cada tarjeta de libro tiene un botón "Agregar al carrito" que suma el ítem a un array `carrito[]` y actualiza un contador visible en el header.
- 02 Un botón "Ver carrito" abre una modal con la lista de ítems, cantidad de cada uno, subtotal por ítem y total general.
- 03 Dentro de esa modal un botón "Comprar" lanza un Wizard de 3 pasos: Paso 1 — datos de envío, Paso 2 — medio de pago, Paso 3 — resumen y confirmación.
- 04 El botón "Siguiente" valida que el paso actual esté completo antes de avanzar; si hay campos vacíos, inserta un mensaje de error con `createElement`.
- 05 Al confirmar en el paso 3: vaciar el carrito, cerrar la modal y mostrar un toast de éxito que desaparece con `setTimeout` a los 4 segundos.

Criterios de evaluación: ¿cómo se que está bien?

Patrones implementados

- Carrito como estado (array)
- Wizard con validación por paso
- Toast con `setTimeout`

Calidad del código

- Funciones separadas por responsabilidad
- Errores con `createElement`
- Estado del carrito consistente

Búsqueda en vivo + acordeón de reseñas + historial

Partiendo del ejercicio anterior resuelto, incorporá tres funcionalidades avanzadas que combinan DOM dinámico, eventos de teclado, timing y ARIA: un buscador en vivo, un acordeón de reseñas dentro de cada tarjeta y un historial de libros visitados.

Requerimientos

- 01 Búsqueda en vivo: un input escucha el evento `input` y filtra en tiempo real las tarjetas por título o autor. Si no hay resultados, insertar un mensaje con `createElement`.

02 Acordeón de reseñas: cada tarjeta tiene una sección colapsable "Ver reseña" que expande el texto con transición CSS. Debe actualizar `aria-expanded` en cada toggle y soportar apertura/cierre con Enter además del clic.

03 Historial de visitas: cada vez que se abre la modal de un libro, ese título se registra en `historial[]`. Un panel "Visitados recientemente" muestra los últimos 5 libros únicos, generado con `createElement`.

04 La búsqueda y los filtros de categoría deben funcionar en conjunto: si hay un filtro activo y el usuario escribe, el resultado es la intersección de ambos.

05 El acordeón cierra automáticamente si otra tarjeta abre el suyo (solo uno abierto a la vez).

Criterios de evaluación: ¿cómo se que está bien?

Integración

- Filtro + búsqueda en conjunto
- Historial como fuente de verdad
- Un solo acordeón abierto a la vez

Accesibilidad y UX

- `aria-expanded` actualizado
- Teclado funcional en acordeón
- Feedback en búsqueda sin resultados